



Controlling Test Execution Order with @Order

Some integration tests require methods to run in a specific order.

Example: **save** → **find** → **update** → **delete** the same database record.

JUnit normally executes tests in a non-deterministic order.

To control the order, we create a new class named **MethodsOrderedByOrderIndexTest**.

Several simple test methods are added for demonstration.

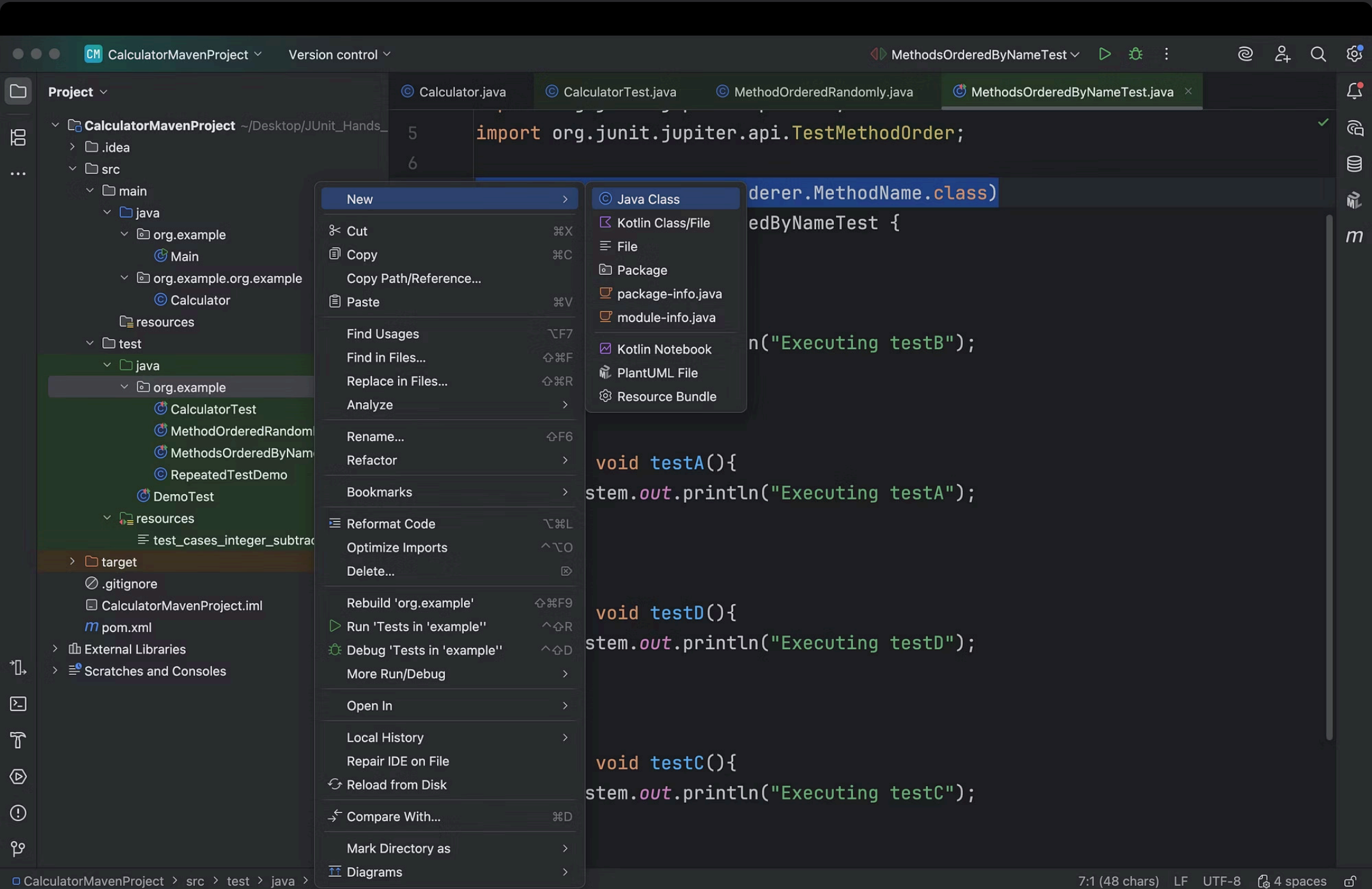


by **Rodrigo Martins Pagliares**

Last update: 12/07/2025

Controlling Test Execution Order with @Order

Let's create a new classe to demonstrate the use of @Order annoation.



Controlling Test Execution Order with @Order (contd.)

Copy the contents of the class created in the previous lesson.

The screenshot shows an IDE window for a project named 'CalculatorMavenProject'. The left sidebar displays the project structure, with the file 'MethodsOrderedByNameTest.java' selected under the path 'src > test > java > org > example'. The main editor area shows the code for 'MethodsOrderedByNameTest.java'. The code includes an import for 'org.junit.jupiter.api.TestMethodOrder', an annotation '@TestMethodOrder(MethodOrderer.MethodName.class)', and a class definition 'MethodsOrderedByNameTest' containing four test methods: 'testB()', 'testA()', 'testD()', and 'testC()'. Each test method prints a message to the console. The status bar at the bottom indicates the file is at line 28, column 6, with 350 characters and 18 line breaks, using LF line endings, UTF-8 encoding, and 4 spaces for indentation.

```
5 import org.junit.jupiter.api.TestMethodOrder;
6
7 @TestMethodOrder(MethodOrderer.MethodName.class)
8 public class MethodsOrderedByNameTest {
9
10     @Test
11     public void testB(){
12         System.out.println("Executing testB");
13     }
14
15     @Test
16     public void testA(){
17         System.out.println("Executing testA");
18     }
19
20     @Test
21     public void testD(){
22         System.out.println("Executing testD");
23     }
24
25     @Test
26     public void testC(){
27         System.out.println("Executing testC");
28     }
29 }
```


Controlling Test Execution Order with @Order (contd.)

Paste the contents in the newly created class

CalculatorMavenProject

Version control

MethodsOrderedByNameTest

MethodsOrderedByOrderIndexTest

Project

CalculatorMavenProject

src

main

java

org.example

org.example.org.example

resources

test

java

org.example

CalculatorTest

MethodOrderedRandomly

MethodsOrderedByNameTest

MethodsOrderedByOrderIndexTest

RepeatedTestDemo

DemoTest

resources

test_cases_integer_subtraction.csv

target

.gitignore

CalculatorMavenProject.iml

pom.xml

External Libraries

Scratches and Consoles

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

import org.junit.jupiter.api.Test;

public class MethodsOrderedByOrderIndexTest {

@Test

public void testB(){

System.out.println("Executing testB");

}

@Test

public void testA(){

System.out.println("Executing testA");

}

@Test

public void testD(){

System.out.println("Executing testD");

}

@Test

public void testC(){

System.out.println("Executing testC");

}

}

CalculatorMavenProject

src

test

java

org

example

MethodsOrderedByOrderIndexTest

testC

24:6

LF

UTF-8

4 spaces

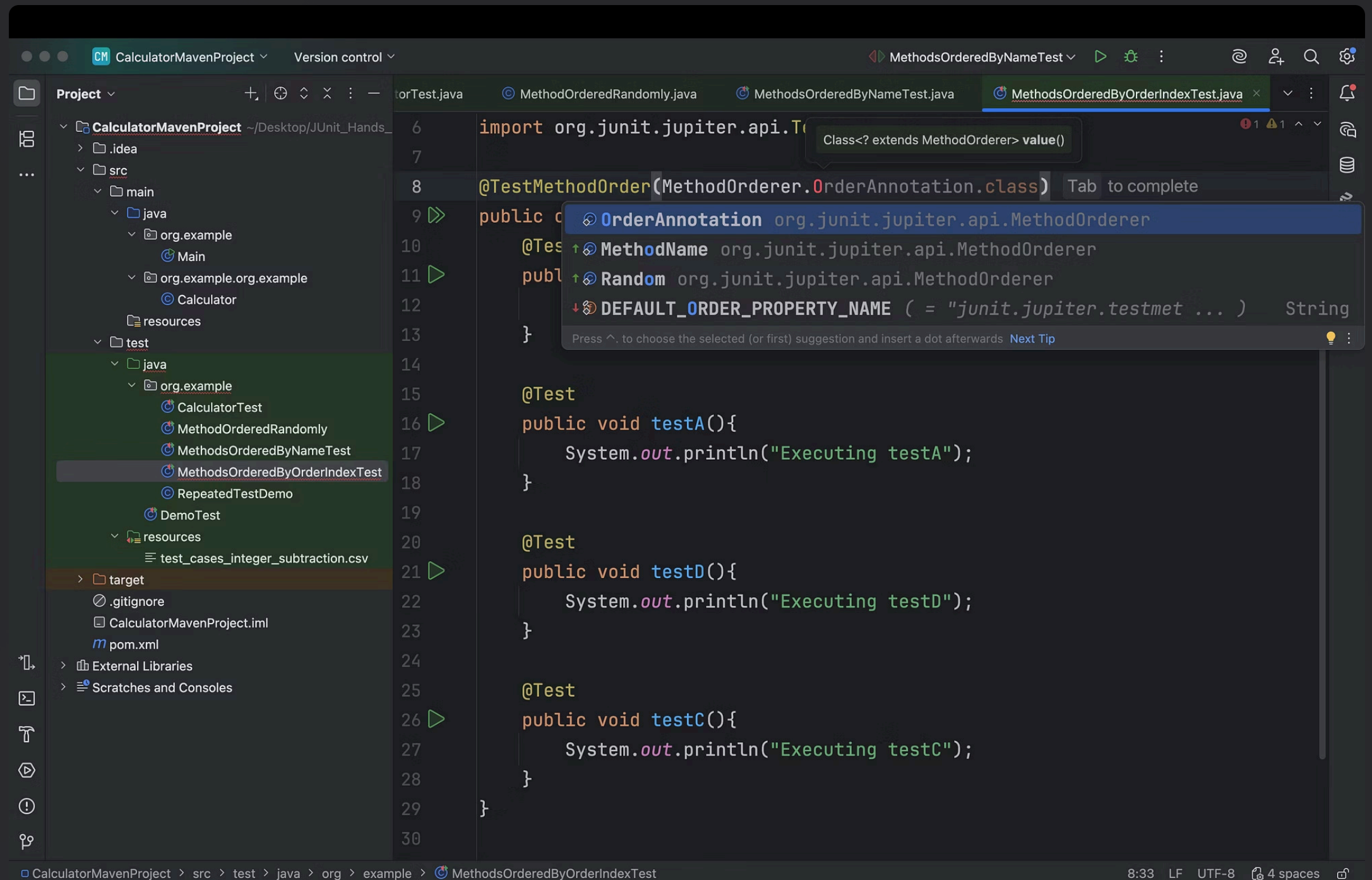
Using @TestMethodOrder + MethodOrderer.OrderAnnotation

- To enforce a **custom execution order**, annotate the class:
`@TestMethodOrder(MethodOrderer.OrderAnnotation.class)`
- Each test method must then receive an `@Order(n)` annotation.
- The `n` value determines the **execution order**.
- **Lower numbers run first**; higher numbers run later.
- Without these annotations, **JUnit** will run tests in arbitrary order.



Controlling Test Execution Order with @Order (contd.)

Use the annotation `@TestMethodOrder` and the interface `MethodOrderer` to start configuring the execution order of the test methods.



The screenshot shows an IDE with a project named 'CalculatorMavenProject'. The left sidebar displays the project structure, including the 'test' directory where the file 'MethodsOrderedByOrderIndexTest.java' is located. The main editor window shows the code for this file, which implements the 'MethodOrderer' interface. The code includes an import for 'org.junit.jupiter.api.TestMethodOrder', an annotation '@TestMethodOrder(MethodOrderer.OrderAnnotation.class)', and three test methods: 'testA()', 'testD()', and 'testC()'. A tooltip is visible over the 'OrderAnnotation' class, showing its inheritance hierarchy and a list of methods: 'methodName()', 'random()', and 'defaultOrderPropertyName()'. The status bar at the bottom indicates the current file is 'MethodsOrderedByOrderIndexTest' and the encoding is 'UTF-8'.

```
import org.junit.jupiter.api.TestMethodOrder;
import org.junit.jupiter.api.MethodOrderer;

@TestMethodOrder(MethodOrderer.OrderAnnotation.class)

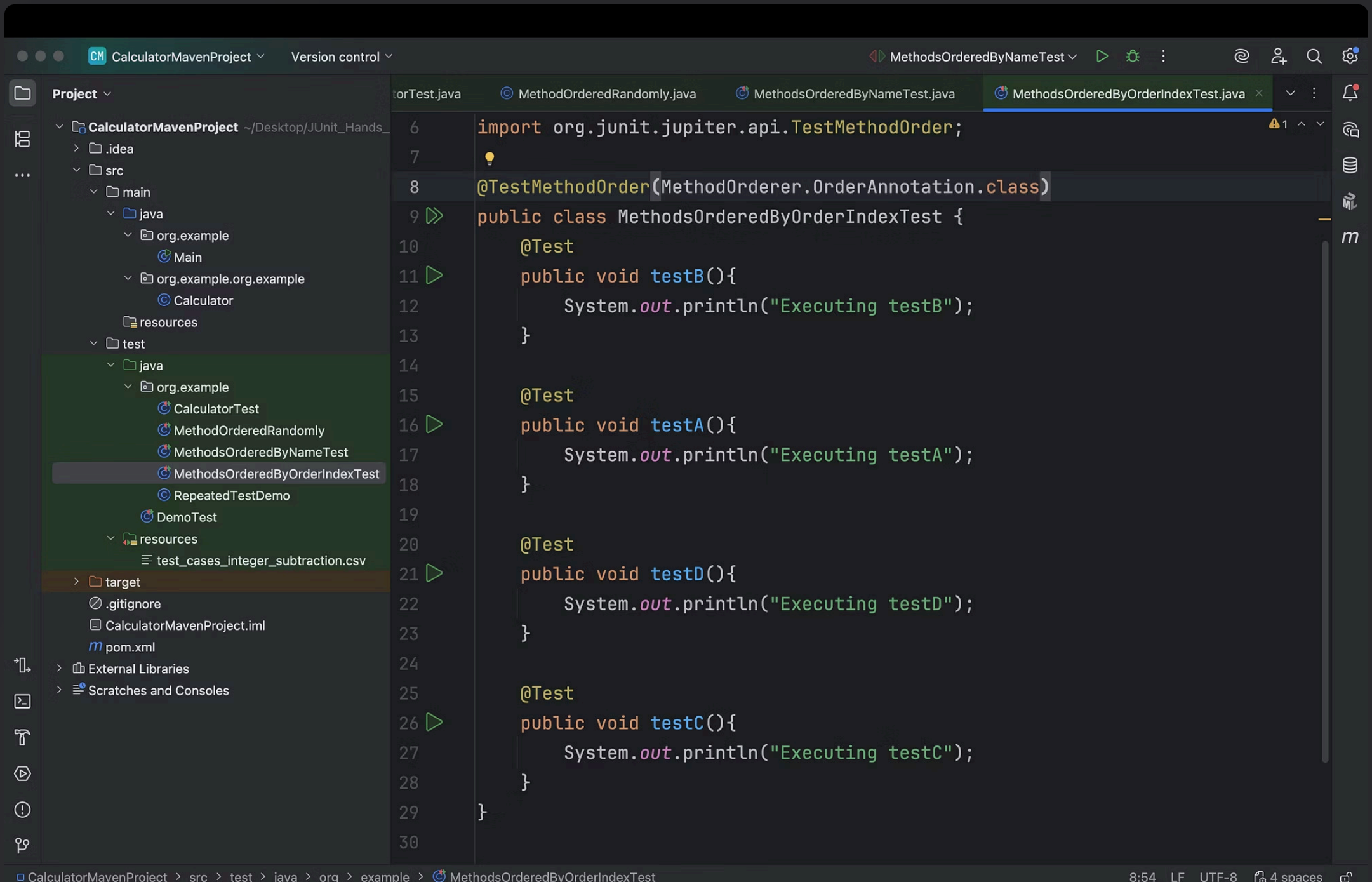
public class MethodsOrderedByOrderIndexTest {

    @Test
    public void testA(){
        System.out.println("Executing testA");
    }

    @Test
    public void testD(){
        System.out.println("Executing testD");
    }

    @Test
    public void testC(){
        System.out.println("Executing testC");
    }
}
```


Controlling Test Execution Order with @Order (contd.)



The screenshot shows an IDE window for a project named "CalculatorMavenProject". The left sidebar displays the project structure, with the file "MethodsOrderedByOrderIndexTest.java" selected under the path "src > test > java > org > example". The main editor area shows the code for this class, which is annotated with `@TestMethodOrder(MethodOrderer.OrderAnnotation.class)`. The code defines a public class `MethodsOrderedByOrderIndexTest` with four test methods: `testB()`, `testA()`, `testD()`, and `testC()`, each printing a message to the console. The line numbers 6 through 30 are visible on the left side of the code editor.

```
6 import org.junit.jupiter.api.TestMethodOrder;
7
8 @TestMethodOrder(MethodOrderer.OrderAnnotation.class)
9
10 public class MethodsOrderedByOrderIndexTest {
11     @Test
12     public void testB(){
13         System.out.println("Executing testB");
14     }
15
16     @Test
17     public void testA(){
18         System.out.println("Executing testA");
19     }
20
21     @Test
22     public void testD(){
23         System.out.println("Executing testD");
24     }
25
26     @Test
27     public void testC(){
28         System.out.println("Executing testC");
29     }
30 }
```

CalculatorMavenProject > src > test > java > org > example > MethodsOrderedByOrderIndexTest 8:54 LF UTF-8 4 spaces

Controlling Test Execution Order with @Order (contd.)

Before testing using `@TestMethodOrder`, let's see what is the output in the console without using this annotation.

CalculatorMavenProject

Version control

MethodsOrderedByOrderIndexTest

Calculator.java

CalculatorTest.java

MethodOrderedRandomly.java

MethodsOrderedByNameTest.java

MethodsOrderedByOrderIndexTest.java

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

//@TestMethodOrder(MethodOrderer.OrderAnnotation.class)

public class MethodsOrderedByOrderIndexTest {

@Test

public void testB(){

System.out.println("Executing testB");

}

@Test

public void testA(){

System.out.println("Executing testA");

}

@Test

public void testD(){

System.out.println("Executing testD");

}

Run

MethodsOrderedByOrderIndexTest

✓ 4 tests passed

4 tests total, 26 ms

✓ testA()

19 ms

✓ testB()

7 ms

✓ testC()

✓ testD()

/Library/Java/JavaVirtualMachines/temurin-25.jdk/Contents/Home/bin/java

Executing testA

Executing testB

Executing testC

Executing testD

CalculatorMavenProject

src

test

java

org

example

MethodsOrderedByOrderIndexTest

6:15

LF

UTF-8

4 spaces

Assigning Order to Test Methods

Methods in the class appear in a **random sequence**: B → A → D → C.

JUnit will **not** respect source order by default.

To **enforce** the desired **order**:

01

@Order(1) → method testB

02

@Order(2) → method testA

03

@Order(3) → method testD

04

@Order(4) → method testC

After annotation, execution order matches the order indices exactly.

Controlling Test Execution Order with @Order (contd.)

Let's now uncomment the `@TestMethodOrder` and resume configuring the method order with `@Order` annotation.

CalculatorMavenProject

Version control

MethodsOrderedByOrderIndexTest

Calculator.java

CalculatorTest.java

MethodOrderedRandomly.java

MethodsOrderedByNameTest.java

MethodsOrderedByOrderIndexTest.java

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

```
@TestMethodOrder(MethodOrderer.OrderAnnotation.class)
public class MethodsOrderedByOrderIndexTest {
    @Order(1)
    @Test
    public void testB(){
        System.out.println("Executing testB");
    }

    @Order(2)
    @Test
    public void testA(){
        System.out.println("Executing testA");
    }

    @Order(3)
    @Test
    public void testC(){
        System.out.println("Executing testC");
    }

    @Order(4)
    @Test
    public void testD(){
        System.out.println("Executing testD");
    }
}
```

Run

MethodsOrderedByOrderIndexTest

✓ 4 tests passed 4 tests total, 26 ms

testA() 7 ms

testB() 17 ms

testC() 2 ms

testD()

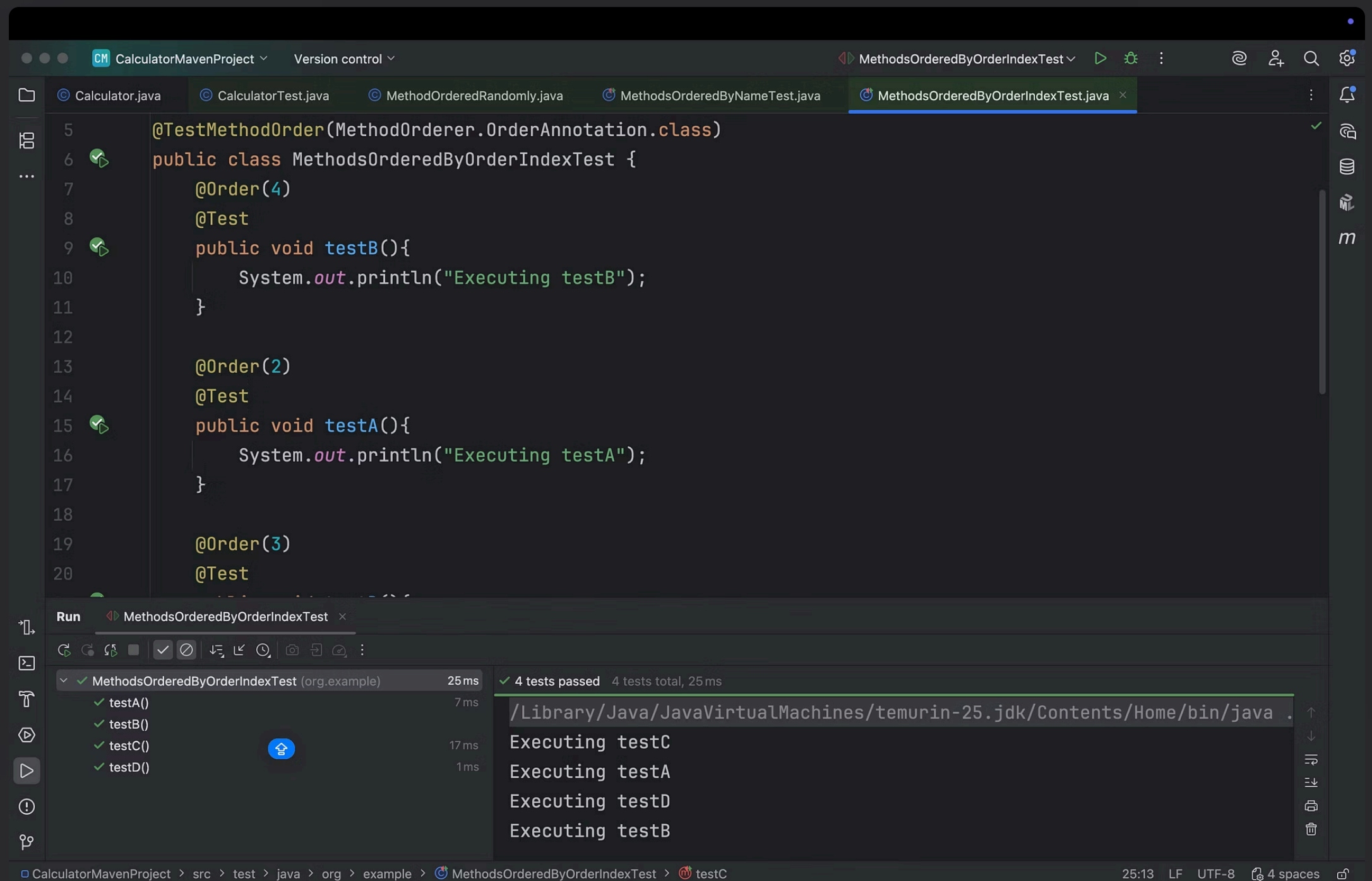
```
/Library/Java/JavaVirtualMachines/temurin-25.jdk/Contents/Home/bin/java .
Executing testB
Executing testA
Executing testD
Executing testC
```

CalculatorMavenProject > src > test > java > org > example > MethodsOrderedByOrderIndexTest > testD

19:14 LF UTF-8 4 spaces

Controlling Test Execution Order with @Order (contd.)

Experiment changing the order and running the test methods again!



The screenshot shows an IDE window with the following components:

- Editor:** Displays the code for `MethodsOrderedByOrderIndexTest`. The code includes annotations `@TestMethodOrder(MethodOrderer.OrderAnnotation.class)` and `@Order` to control the execution order of the test methods.
- Run Panel:** Shows the execution results of the test class. It indicates that 4 tests passed (testA, testB, testC, testD) in a total of 25 ms.
- Output Console:** Displays the output of the test execution, showing the sequence of test methods executed: testC, testA, testD, and testB.

```
5 @TestMethodOrder(MethodOrderer.OrderAnnotation.class)
6 public class MethodsOrderedByOrderIndexTest {
7     @Order(4)
8     @Test
9     public void testB(){
10         System.out.println("Executing testB");
11     }
12
13     @Order(2)
14     @Test
15     public void testA(){
16         System.out.println("Executing testA");
17     }
18
19     @Order(3)
20     @Test
21     public void testC(){
22         System.out.println("Executing testC");
23     }
24     @Test
25     public void testD(){
26         System.out.println("Executing testD");
27     }
28 }
```

Run Results:

Test Method	Duration
testA()	7 ms
testB()	17 ms
testC()	1 ms
testD()	1 ms

Output Console:

```
/Library/Java/JavaVirtualMachines/temurin-25.jdk/Contents/Home/bin/java .
Executing testC
Executing testA
Executing testD
Executing testB
```

After running the test class, one can see the execution order is now controlled.

This confirms that `@Order` controls the test flow.

Useful for multi-step integration flows where state must be consistent.

Conclusion

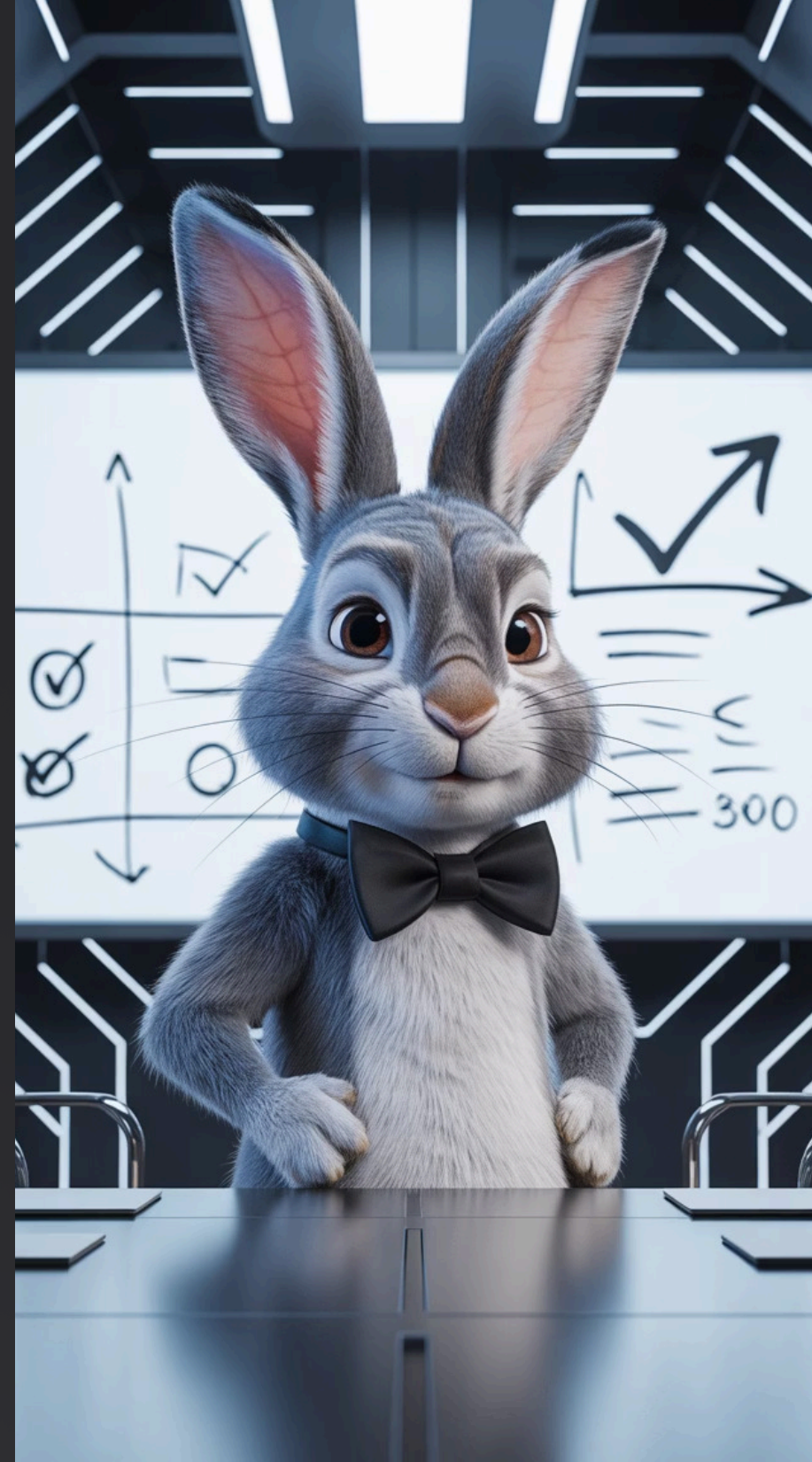
Use

`@TestMethodOrder(MethodOrderer.OrderAnnotation.class)` when test order matters.

Annotate each test with `@Order(n)` to define precise execution flow.

However, avoid test interdependencies unless absolutely necessary.

Ideally, tests should still pass even if order is random.



Open Floor Discussion

- ☐ When is ordering truly necessary vs. a sign of test coupling?
- ☐ Could test fixtures or setup methods reduce the need for ordering?
- ☐ How can ordered tests affect refactoring or long-term maintainability?

